Final project: Annotated R analysis companion

Chunlei Li

Scope

This companion exposes the complete executable code behind the final report. All imports begin with unchanged instructor CSV files. This annotated analysis companion generates the tables and figures; the final Methods, Results and References narrative must also be submitted as required by the course.

```
# All cleaning starts from the three unchanged instructor CSV files.
# Check dependencies before loading packages; install missing packages separately.
required_packages <- c("ggplot2", "dplyr", "tidyr", "patchwork", "rlang",
                       "knitr", "rmarkdown")
missing_packages <- required_packages[!vapply(required_packages, requireNamespace,
                                              logical(1), quietly = TRUE)]
if (length(missing_packages) > 0) {
  stop("Install these packages before knitting: ", paste(missing_packages, collapse = ", "))
}
library(ggplot2)
library(dplyr)
library(tidyr)
library(patchwork)
dir.create("results", showWarnings = FALSE)
dir.create("figures", showWarnings = FALSE)

cov_path <- "E:/ /raw/GSE85307_covar-1.csv"
if (!file.exists(cov_path)) cov_path <- "data/raw/GSE85307_covar.csv"
expr_path <- "E:/ /raw/GSE85307_geneExpression.csv"
gene_path <- "E:/ /raw/GSE85307_geneNames.csv"
# Stop early and identify missing files rather than failing midway through import.
input_paths <- c(cov_path, expr_path, gene_path)
missing_files <- input_paths[!file.exists(input_paths)]
if (length(missing_files) > 0) {
  stop("Missing input files: ", paste(missing_files, collapse = ", "),
       ". Keep this Rmd in the project root with data/raw underneath it.")
}
covar_raw <- read.csv(cov_path, stringsAsFactors = FALSE, check.names = FALSE)
expr_raw <- read.csv(expr_path,
                     row.names = 1, check.names = FALSE)
link_raw <- read.csv(gene_path,
                     stringsAsFactors = FALSE, check.names = FALSE)

# Remove the label before the FIRST colon, then trim surrounding whitespace.
strip_label <- function(x) trimws(sub("^[^:]*:\\s*", "", x, perl = TRUE))
clean <- covar_raw
for (v in setdiff(names(clean), c("subjectID", "geoID"))) {
  clean[[v]] <- strip_label(clean[[v]])
}
num_vars <- c("mat_age_yrs", "delivery_gestAge_wks",
```

```
              "sampleCollection_gestAge_wks", "mat_BMI", "mat_vitD")
for (v in num_vars) {
  before <- clean[[v]]
  clean[[v]] <- suppressWarnings(as.numeric(before))
  if (any(is.na(clean[[v]]) & !is.na(before))) stop("Numeric conversion failed: ", v)
}
# Sample identifiers must be present, nonblank and unique before matching.
stopifnot(!anyNA(clean$geoID), !anyNA(clean$subjectID),
          all(nzchar(trimws(clean$geoID))), all(nzchar(trimws(clean$subjectID))))
stopifnot(!anyDuplicated(clean$geoID), !anyDuplicated(clean$subjectID),
          !anyDuplicated(colnames(expr_raw)), !anyDuplicated(rownames(expr_raw)),
          !anyDuplicated(link_raw$PROBEID), !anyDuplicated(link_raw$GENE),
          setequal(clean$geoID, colnames(expr_raw)),
          all(clean$preg_preeclampsia %in% c("Normal", "Preeclampsia")),
          all(clean$mat_asthma %in% c("0", "1")))
expr <- as.matrix(expr_raw)
stopifnot(is.numeric(expr), all(is.finite(expr)))

# Match expression columns by ID: never assume the input order is the same.
expr <- expr[, match(clean$geoID, colnames(expr)), drop = FALSE]
stopifnot(identical(colnames(expr), clean$geoID))
map <- link_raw[match(rownames(expr), as.character(link_raw$PROBEID)), ]
stopifnot(!anyNA(map$GENE), identical(as.character(map$PROBEID), rownames(expr)))
rownames(expr) <- map$GENE

# Study-specific threshold from Mirzakhani et al. (2016), NOT a diagnostic rule.
clean <- clean %>% mutate(
  preg_preeclampsia = factor(preg_preeclampsia, levels = c("Normal", "Preeclampsia")),
  vitD_group = factor(ifelse(mat_vitD < 30, "<30 ng/mL", ">=30 ng/mL"),
                      levels = c("<30 ng/mL", ">=30 ng/mL")),
  mat_asthma = factor(mat_asthma, levels = c("0", "1"), labels = c("No", "Yes")),
  batch_Microarray = factor(batch_Microarray))
# Preserve all 157 samples; do not remove outliers just to improve the plot.
all_genes <- as.data.frame(t(expr), check.names = FALSE)
analysis_data <- bind_cols(clean, all_genes)
selected_genes <- c("CYP24A1", "PTX3", "CXCR2")
heat_genes <- c(selected_genes, "JAK2", "LYN", "SOCS5", "IL7", "IL2", "TGM2", "TIMP1")
stopifnot(all(heat_genes %in% rownames(expr)), nrow(analysis_data) == 157L)
missing_table <- data.frame(variable = names(clean), missing = colSums(is.na(clean)))
# This analysis retains all samples and does not silently impute missing values.
stopifnot(sum(missing_table$missing) == 0,
          all(vapply(clean[num_vars], function(x) all(is.finite(x)), logical(1))))
write.csv(missing_table, "results/missingness.csv", row.names = FALSE)
write.csv(map, "results/probe_gene_mapping_used.csv", row.names = FALSE)
write.csv(clean, "results/clean_covariates.csv", row.names = FALSE)
write.csv(as.data.frame(table(clean$preg_preeclampsia, clean$vitD_group)),
          "results/group_counts.csv", row.names = FALSE)
write.csv(data.frame(file = input_paths,
                     md5 = unname(tools::md5sum(input_paths))),
          "results/input_checksums.csv", row.names = FALSE)
```

Reusable plotting function

```
# Return a named list of three ggplot objects for each requested gene.
plot_gene_panels <- function(data, genes, continuous, categorical) {
  genes <- unlist(genes, use.names = FALSE)
  if (!is.character(continuous) || anyNA(continuous) ||
      !is.character(categorical) || anyNA(categorical) ||
      anyNA(genes) || anyDuplicated(categorical)) {
    stop("Use nonmissing column names and two distinct categorical variables.")
  }
  if (!is.data.frame(data) || !is.character(genes) || length(genes) < 1L ||
      length(continuous) != 1L || length(categorical) != 2L) {
    stop("Supply a data frame, >=1 gene, one continuous name, and two categorical names.")
  }
  if (anyDuplicated(genes)) stop("Gene names must be unique.")
  if (!all(c(genes, continuous, categorical) %in% names(data))) stop("Unknown column name.")
  if (!is.numeric(data[[continuous]]) || !all(vapply(data[genes], is.numeric, logical(1)))) {
    stop("Gene expression and the continuous covariate must be numeric.")
  }
  if (!all(vapply(data[categorical], function(x) is.factor(x) || is.character(x),
   ↪  logical(1)))) {
    stop("Categorical covariates must be factors or character columns.")
  }
  out <- list()
  for (gene in genes) {
    # Filter only the columns needed for this gene and state the plotted n.
    d <- data[complete.cases(data[, c(gene, continuous, categorical)]), , drop = FALSE]
    numeric_cols <- c(gene, continuous)
    if (!all(vapply(d[numeric_cols], function(x) all(is.finite(x)), logical(1)))) {
      stop("Nonfinite values in expression or continuous covariate for ", gene)
    }
    if (length(unique(d[[continuous]])) < 2L) {
      stop("The continuous covariate must have at least two distinct values.")
    }
    if (nrow(d) < 3L) stop("Too few complete observations for ", gene)
    xlab <- if (continuous == "mat_vitD") "Baseline 25(OH)D (ng/mL)" else continuous
    ylab <- paste0(gene, " expression (provided scale)")
    n_txt <- paste("n =", nrow(d))
    style <- theme_classic(base_size = 11, base_family = "sans") +
      theme(plot.title = element_text(face = "bold", size = 13),
            plot.subtitle = element_text(size = 10, colour = "grey35"),
            legend.position = "top", legend.title = element_blank(),
            strip.background = element_rect(fill = "grey94", colour = NA),
            strip.text = element_text(face = "bold"),
            plot.margin = margin(10, 14, 10, 10))
    hist <- ggplot(d, aes(x = .data[[gene]])) +
      geom_histogram(bins = 24, fill = "#337C8E", colour = "white", linewidth = 0.25) +
      labs(title = paste(gene, "expression distribution"), subtitle = n_txt,
           x = ylab, y = "Number of participants") + style
    scatter <- ggplot(d, aes(x = .data[[continuous]], y = .data[[gene]])) +
      geom_point(colour = "#337C8E", alpha = 0.65, size = 1.7) +
      geom_smooth(method = "lm", formula = y ~ x, se = TRUE,
                  colour = "#B65E27", fill = "#E4B48B", linewidth = 0.8) +
      labs(title = paste(gene, "and", if (continuous == "mat_vitD") "baseline vitamin D" else
       ↪  continuous), subtitle = n_txt,
```

```
          x = xlab, y = ylab) + style
    # Use both categories: x = outcome; facet and colour = vitamin D group.
    counts <- d %>% group_by(across(all_of(categorical))) %>% summarise(n = n(), .groups =
 ↪  "drop")
    ymax <- max(d[[gene]])
    span <- diff(range(d[[gene]])); if (span == 0) span <- 1
    counts$label_y <- ymax + 0.08 * span
    box <- ggplot(d, aes(x = .data[[categorical[1]]], y = .data[[gene]],
                         fill = .data[[categorical[2]]])) +
      geom_boxplot(width = 0.55, outlier.shape = NA, alpha = 0.65, linewidth = 0.45) +
      geom_point(position = position_jitter(width = 0.12, height = 0, seed = 103),
                 shape = 21, size = 1.3, alpha = 0.6, stroke = 0.2) +
      geom_text(data = counts, aes(x = .data[[categorical[1]]], y = label_y,
                                   label = paste0("n = ", n)),
                inherit.aes = FALSE, size = 3.1) +
      facet_wrap(vars(!!rlang::sym(categorical[2]))) +
      scale_fill_manual(values = rep(c("#337C8E", "#D5A13B", "#7D5596", "#AD4E32", "#57A773",
       ↪  "#C27D9F"),
        length.out = length(unique(d[[categorical[2]]])))) +
      scale_y_continuous(expand = expansion(mult = c(0.07, 0.17))) +
      labs(title = paste(gene, "by", paste(categorical, collapse = " and ")), subtitle =
       ↪  n_txt,
           x = if (categorical[1] == "preg_preeclampsia") "Pregnancy outcome" else
            ↪  categorical[1], y = ylab) + style + theme(legend.position = "none")
    out[[gene]] <- list(histogram = hist, scatterplot = scatter, boxplot = box)
  }
  out
}

# Save vector PDF and 300-dpi PNG versions for manuscripts and presentations.
save_gene_panels <- function(panels) {
  for (gene in names(panels)) for (kind in names(panels[[gene]])) {
    base <- file.path("figures", paste(gene, kind, sep = "_"))
    w <- if (kind == "boxplot") 8.2 else 7
    ggsave(paste0(base, ".pdf"), panels[[gene]][[kind]], width = w, height = 4.7)
    ggsave(paste0(base, ".png"), panels[[gene]][[kind]], width = w, height = 4.7, dpi = 300)
  }
}
```

```
panels <- plot_gene_panels(analysis_data, selected_genes, "mat_vitD",
                           c("preg_preeclampsia", "vitD_group"))
save_gene_panels(panels)
```

Summary table, heatmap, ECDF and software citations

```
# Table 1: three continuous covariates, stratified by pregnancy outcome.
cont <- c("mat_vitD", "mat_age_yrs", "mat_BMI")
labels <- c("25(OH)D (ng/mL)", "Age (years)", "BMI (kg/m2)")
summary_table <- data.frame(Characteristic = c(
  as.vector(rbind(paste0(labels, ": mean (SD)"), paste0(labels, ": median [Q1, Q3]"))),
  "25(OH)D <30 ng/mL, n (%)", "25(OH)D >=30 ng/mL, n (%)"), check.names = FALSE)
for (grp in c("Overall", levels(clean$preg_preeclampsia))) {
```

```r
  d <- if (grp == "Overall") clean else filter(clean, preg_preeclampsia == grp)
  vals <- character()
  for (v in cont) {
    z <- d[[v]]; q <- quantile(z, c(.25, .5, .75))
    vals <- c(vals, sprintf("%.2f (%.2f)", mean(z), sd(z)),
              sprintf("%.2f [%.2f, %.2f]", q[2], q[1], q[3]))
  }
  counts <- table(d$vitD_group)
  vals <- c(vals, sprintf("%d (%.1f)", counts, 100 * counts / nrow(d)))
  summary_table[[paste0(grp, " (n=", nrow(d), ")")]] <- vals
}
write.csv(summary_table, "results/Table1.csv", row.names = FALSE)

# Descriptive summaries: genes are not chosen by p-value.
gene_summary <- bind_rows(lapply(selected_genes, function(g) {
  y <- analysis_data[[g]]
  fit <- lm(y ~ mat_vitD, data = analysis_data)
  ci <- confint(fit)[2, ]
  data.frame(gene = g, n = length(y), mean = mean(y), sd = sd(y),
             median = median(y), Q1 = quantile(y, .25), Q3 = quantile(y, .75),
             min = min(y), max = max(y),
             pearson_r_descriptive = cor(analysis_data$mat_vitD, y),
             slope_per_10ng = 10 * coef(fit)[2], CI_low = 10 * ci[1], CI_high = 10 * ci[2])
}))
write.csv(gene_summary, "results/gene_summaries.csv", row.names = FALSE)
group_gene_summary <- analysis_data %>% group_by(preg_preeclampsia, vitD_group) %>%
  summarise(n = n(), across(all_of(selected_genes),
    list(median = median, Q1 = ~quantile(.x, .25), Q3 = ~quantile(.x, .75))), .groups =
    ↪ "drop")
write.csv(group_gene_summary, "results/gene_group_summaries.csv", row.names = FALSE)

# Ten fixed available genes; row-wise z-scores; Euclidean/complete clustering.
m <- expr[heat_genes, , drop = FALSE]
stopifnot(all(apply(m, 1, sd) > 0))
z <- t(scale(t(m)))
row_hc <- hclust(dist(z), method = "complete")
col_hc <- hclust(dist(t(z)), method = "complete")
gene_order <- rownames(z)[row_hc$order]
sample_order <- colnames(z)[col_hc$order]
heat_df <- as.data.frame(as.table(z), stringsAsFactors = FALSE)
names(heat_df) <- c("gene", "geoID", "z")
heat_df$x <- match(heat_df$geoID, sample_order)
heat_df$gene <- factor(heat_df$gene, levels = rev(gene_order))
# Convert sample clustering to dendrogram segments aligned with heatmap columns.
segments <- data.frame(x = numeric(), xend = numeric(), y = numeric(), yend = numeric())
leaf_positions <- match(seq_along(col_hc$order), col_hc$order)
walk <- function(node) {
  if (node < 0) return(c(leaf_positions[-node], 0))
  a <- walk(col_hc$merge[node, 1]); b <- walk(col_hc$merge[node, 2])
  h <- col_hc$height[node]
  segments <<- rbind(segments,
    data.frame(x = c(a[1], b[1], a[1]), xend = c(a[1], b[1], b[1]),
               y = c(a[2], b[2], h), yend = c(h, h, h)))
  c(mean(c(a[1], b[1])), h)
}
invisible(walk(nrow(col_hc$merge)))
```

```r
xscale <- scale_x_continuous(limits = c(0.5, ncol(z) + 0.5), expand = c(0, 0))
dend <- ggplot(segments) + geom_segment(aes(x=x, xend=xend, y=y, yend=yend),
  linewidth = .25, colour = "grey30") + xscale + theme_void()
ann <- clean[match(sample_order, clean$geoID), c("geoID", "preg_preeclampsia", "vitD_group")]
stopifnot(identical(ann$geoID, sample_order))
ann$x <- seq_len(nrow(ann))
ann_long <- pivot_longer(ann, cols = c(preg_preeclampsia, vitD_group),
                         names_to = "track", values_to = "value")
ann_long$track <- factor(ann_long$track, levels = c("vitD_group", "preg_preeclampsia"),
                         labels = c("Vitamin D", "Outcome"))
tracks <- ggplot(ann_long, aes(x, track, fill = value)) + geom_tile() + xscale +
  scale_fill_manual(values = c("Normal"="#4393A7", "Preeclampsia"="#B55F38",
                               "<30 ng/mL"="#595959", ">=30 ng/mL"="#D5A13B")) +
  labs(x = NULL, y = NULL, fill = "Sample annotations") + theme_minimal(base_size = 10) +
  theme(panel.grid = element_blank(), axis.text.x = element_blank(),
        legend.position = "right", plot.margin = margin(0, 5, 0, 5))
heat <- ggplot(heat_df, aes(x, gene, fill = z)) + geom_tile() + xscale +
  scale_fill_gradient2(low = "#2166AC", mid = "#FAFAFA", high = "#B2182B",
                       midpoint = 0, name = "Gene-wise\nz-score") +
  labs(x = "Participants in hierarchical cluster order", y = NULL) +
  theme_minimal(base_size = 10) +
  theme(panel.grid = element_blank(), axis.text.x = element_blank(),
        axis.ticks.x = element_blank(), legend.position = "right")
heatmap_plot <- (dend / tracks / heat) + plot_layout(heights = c(1.2, .65, 4)) +
  plot_annotation(title = "Ten-gene expression profile in early pregnancy",
                  theme = theme(plot.title = element_text(face = "bold", size = 14)))
ggsave("figures/heatmap.pdf", heatmap_plot, width = 10, height = 6.4)
ggsave("figures/heatmap.png", heatmap_plot, width = 10, height = 6.4, dpi = 300)
write.csv(data.frame(cluster_position = seq_along(sample_order), geoID = sample_order),
          "results/heatmap_sample_order.csv", row.names = FALSE)

# Additional candidate: empirical cumulative distribution function (ECDF).
ecdf_data <- analysis_data %>% select(geoID, preg_preeclampsia, vitD_group,
 ↪  all_of(selected_genes)) %>%
  pivot_longer(all_of(selected_genes), names_to = "gene", values_to = "expression") %>%
  mutate(group = interaction(preg_preeclampsia, vitD_group, sep = " | "))
ecdf_n <- ecdf_data %>% distinct(geoID, group) %>% count(group)
legend_labels <- setNames(paste0(ecdf_n$group, " (n=", ecdf_n$n, ")"), ecdf_n$group)
ecdf_plot <- ggplot(ecdf_data, aes(expression, colour = group)) +
  stat_ecdf(geom = "step", linewidth = .75, pad = FALSE) +
  facet_wrap(~ gene, scales = "free_x", ncol = 1) +
  scale_colour_manual(values = c("#337C8E", "#AD4E32", "#D5A13B", "#7D5596"),
                      labels = legend_labels) +
  scale_y_continuous(labels = function(x) paste0(100*x, "%"), breaks = c(0,.25,.5,.75,1)) +
  labs(title = "Comparing the full expression distributions",
       x = "Gene expression (provided scale)",
       y = "Participants at or below this expression", colour = NULL) +
  theme_classic(base_size = 11) + theme(legend.position = "bottom",
    legend.text = element_text(size = 8), plot.title = element_text(face = "bold"),
    strip.background = element_rect(fill = "grey94", colour = NA)) +
  guides(colour = guide_legend(nrow = 2))
ggsave("figures/additional_ECDF.pdf", ecdf_plot, width = 8.5, height = 8)
ggsave("figures/additional_ECDF.png", ecdf_plot, width = 8.5, height = 8, dpi = 300)

# Record exact versions and actual package citations from this runtime.
packages_used <- c("ggplot2", "dplyr", "tidyr", "patchwork", "rlang", "knitr", "rmarkdown")
```

```r
write.csv(data.frame(software = c("R", packages_used), version =
  c(as.character(getRversion()),
  vapply(packages_used, function(x) as.character(packageVersion(x)), character(1)))),
  "results/software_versions.csv", row.names = FALSE)
writeLines(capture.output(sessionInfo()), "results/sessionInfo.txt")
bib <- "references.bib"
# Make the bibliography self-contained; no external .bib input is required.
writeLines(c(
  "@article{Mirzakhani2016,",
  "  author = {Mirzakhani, H. and others},",
  "  title = {Early pregnancy vitamin D status and risk of preeclampsia},",
  "  journal = {Journal of Clinical Investigation},",
  "  year = {2016}, volume = {126}, number = {12}, pages = {4702--4715},",
  "  doi = {10.1172/JCI89031}",
  "}"
), bib)
refs <- list(Rbase = citation())
for (pkg in packages_used) refs[[paste0("pkg", gsub("[^A-Za-z0-9]", "", pkg))]] <-
  citation(pkg)
for (key in names(refs)) {
  entries <- refs[[key]]
  for (j in seq_along(entries)) {
    b <- toBibtex(entries[j])
    b[1] <- sub("\\{[^,]*,", paste0("{", key, if(j > 1) paste0(j) else "", ","), b[1])
    cat(paste(b, collapse="\n"), "\n\n", file = bib, append = TRUE)
  }
}
```

Key outputs

```r
knitr::kable(summary_table, caption = "Participant characteristics overall and by pregnancy
  outcome. Continuous variables are summarized as mean (SD) and median [Q1, Q3]; vitamin D
  groups as n (%).")
```

Table 1: Participant characteristics overall and by pregnancy outcome. Continuous variables are summarized as mean (SD) and median [Q1, Q3]; vitamin D groups as n (%).

Characteristic	Overall (n=157)	Normal (n=110)	Preeclampsia (n=47)
25(OH)D (ng/mL): mean (SD)	22.98 (13.15)	24.37 (14.55)	19.72 (8.33)
25(OH)D (ng/mL): median [Q1, Q3]	20.40 [13.40, 29.50]	21.80 [13.85, 32.08]	17.70 [12.55, 27.40]
Age (years): mean (SD)	26.38 (5.04)	26.63 (5.08)	25.78 (4.95)
Age (years): median [Q1, Q3]	26.57 [22.15, 30.14]	26.87 [22.23, 30.64]	25.72 [21.73, 28.88]
BMI (kg/m2): mean (SD)	28.94 (7.88)	27.99 (7.66)	31.16 (8.03)
BMI (kg/m2): median [Q1, Q3]	27.33 [22.90, 33.27]	26.54 [22.35, 30.97]	31.24 [24.62, 36.41]
25(OH)D <30 ng/mL, n (%)	119 (75.8)	78 (70.9)	41 (87.2)
25(OH)D >=30 ng/mL, n (%)	38 (24.2)	32 (29.1)	6 (12.8)

```
knitr::kable(gene_summary[, c("gene", "n", "median", "pearson_r_descriptive")], row.names =
↪  FALSE)
```

gene	n	median	pearson_r_descriptive
CYP24A1	157	4.789505	-0.0566412
PTX3	157	4.867430	-0.0349912
CXCR2	157	12.196606	0.3459991

Final figures

The following figures retain the supplied expression scale. The fitted line is unadjusted; its shaded band is a model-based 95% confidence interval for the fitted mean, not a prediction interval.

```
panels[["CYP24A1"]][["histogram"]]
```

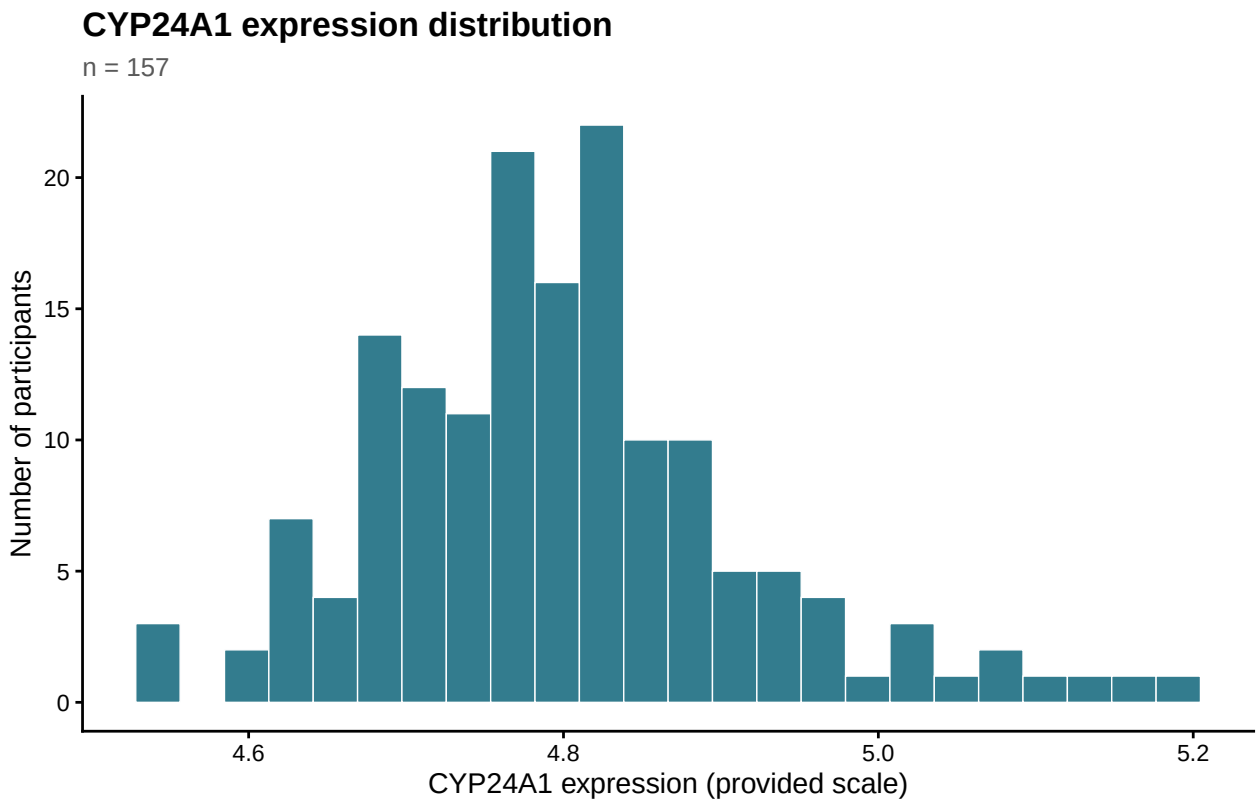


Figure 1: Distribution of CYP24A1 expression using 24 bins.

```
panels[["CYP24A1"]][["scatterplot"]]
```

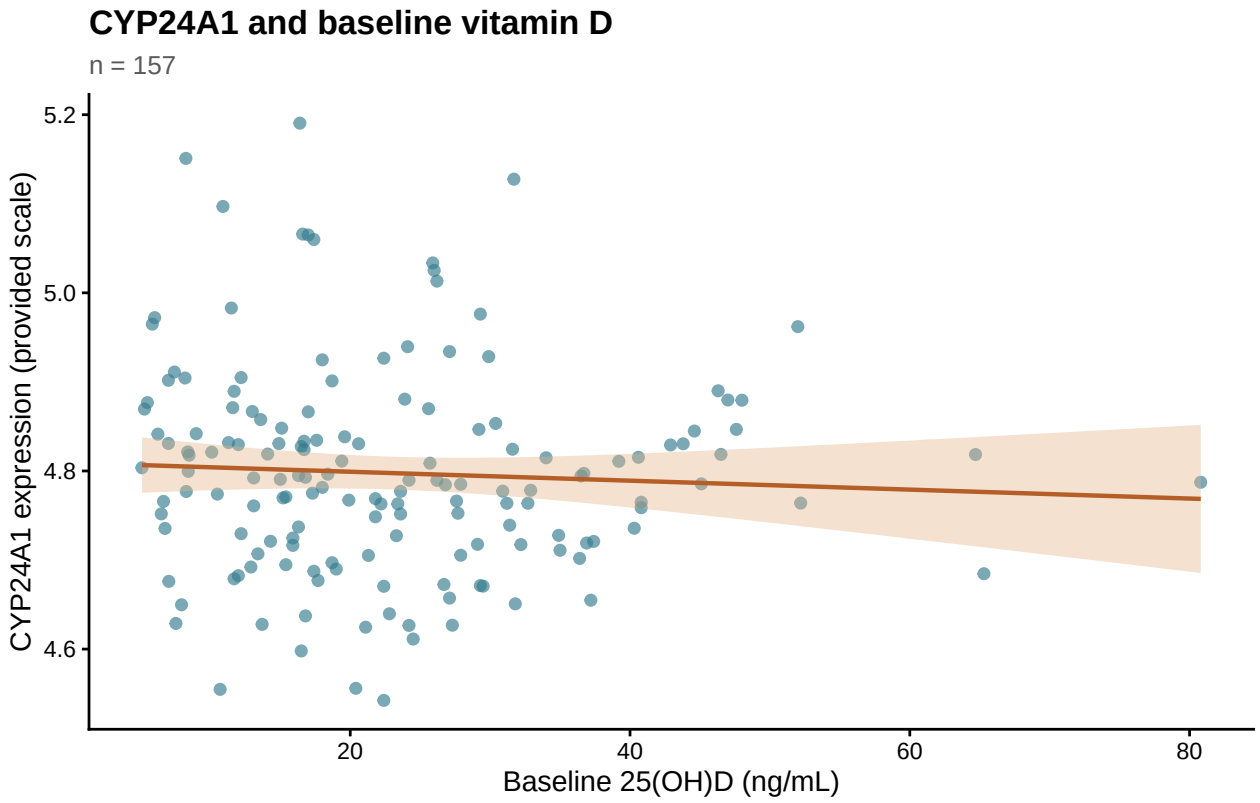


Figure 2: Baseline vitamin D and CYP24A1 expression, with an unadjusted linear fit and 95% confidence band.

```
panels[["CYP24A1"]][["boxplot"]]
```

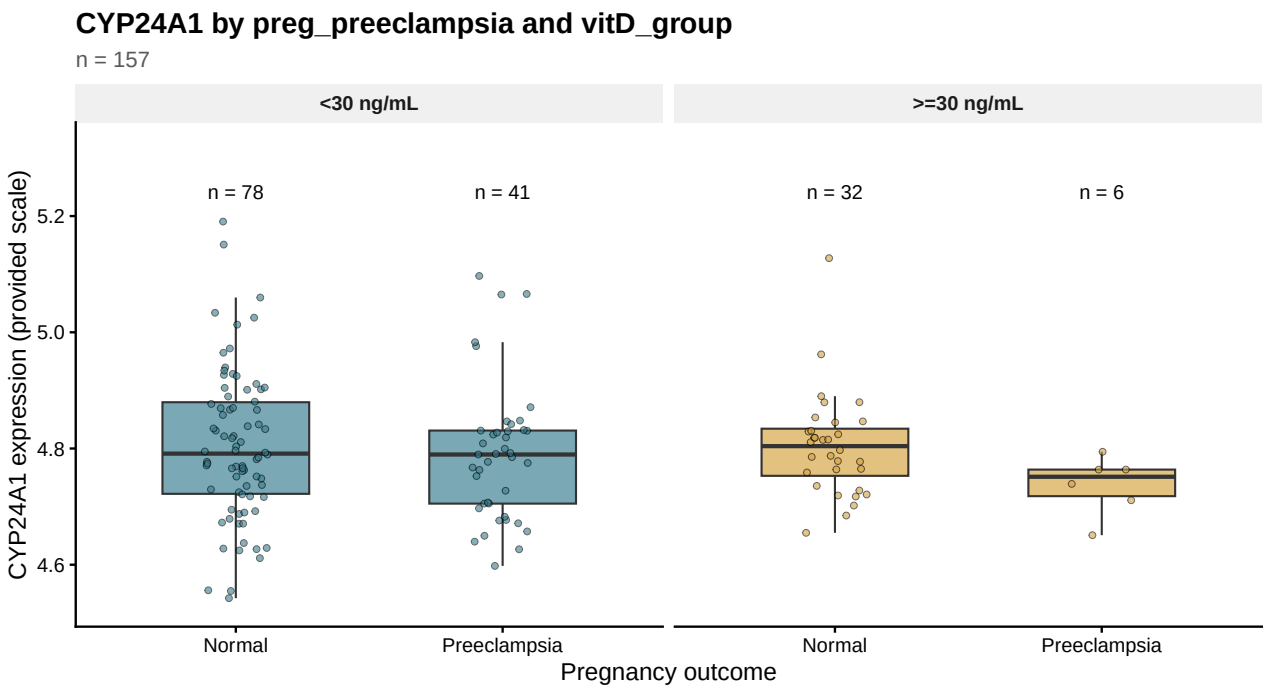


Figure 3: CYP24A1 expression by pregnancy outcome and vitamin D group. Boxes show median and IQR; whiskers reach observed values within 1.5 IQR of the hinges. All observations and subgroup counts are displayed; jitter is horizontal only.

```
heatmap_plot
```

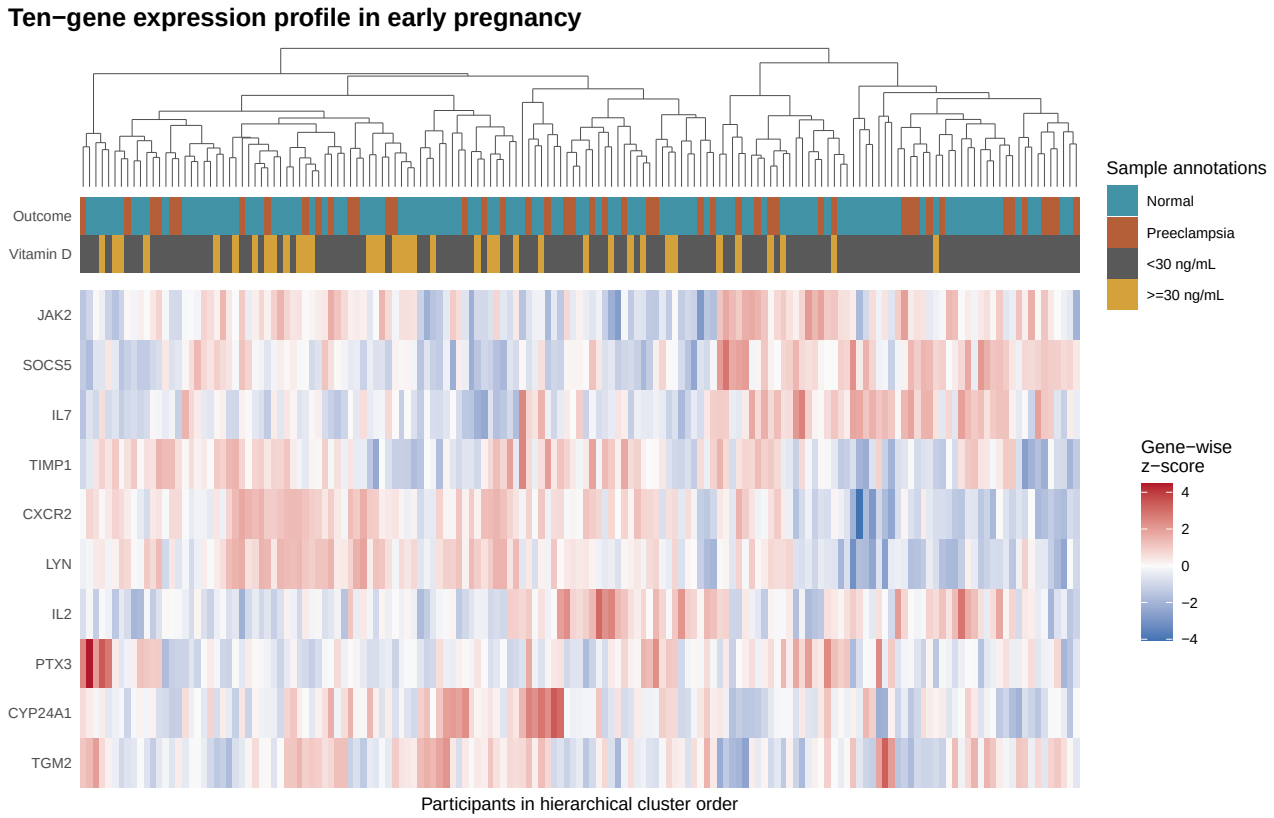


Figure 4: Ten-gene heatmap with pregnancy outcome and vitamin D annotations. Expression is standardized within each gene. Both genes and participants are ordered by hierarchical clustering using Euclidean distance and complete linkage; the participant dendrogram is displayed. Colors compare relative expression within genes, not absolute expression between genes.

```
ecdf_plot
```

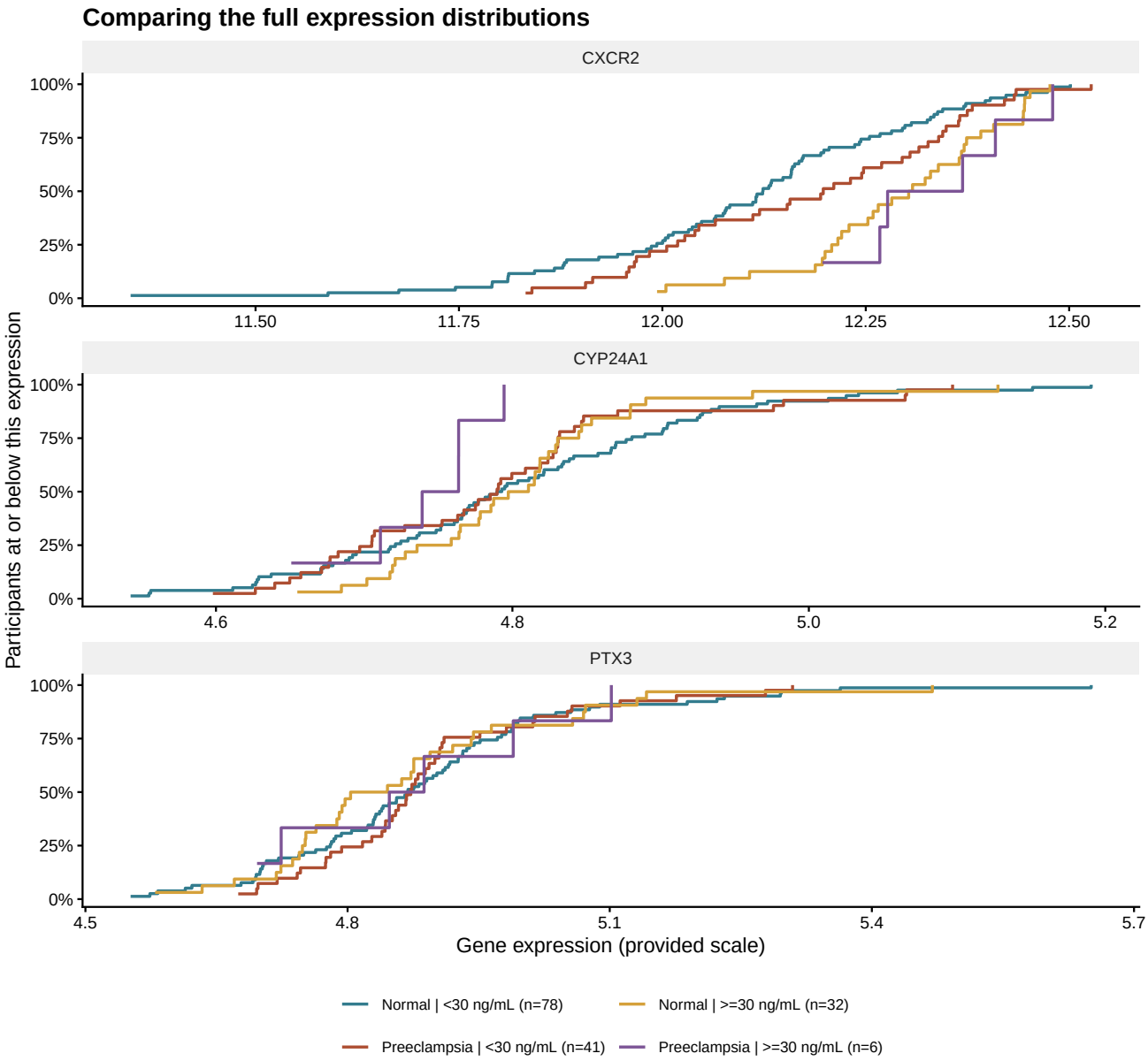


Figure 5: Empirical cumulative expression distributions for three genes by outcome and vitamin D group. At each x value, the curve gives the fraction of subgroup participants with expression at or below x. The higher-vitamin-D preeclampsia group has only six participants.

Interpretation and limitations

These figures describe exploratory, unadjusted associations and do not establish causal effects. Distributions may overlap even when their centers differ; visual comparisons do not replace formal inference. The subgroup with six participants has limited precision. Heatmap clusters reflect similarity under the chosen scaling, genes and distance method; they are not validated disease subtypes. Vitamin D grouping is derived from the continuous vitamin D variable and is not an independent exposure. The ECDF is an additional-figure candidate: confirm its eligibility with the teaching team before final submission.

Software environment

```
# Display versions in the companion and also save them in results/sessionInfo.txt.
sessionInfo()
```

```
#> R version 4.6.1 (2026-06-24 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 10 x64 (build 19045)
#>
#> Matrix products: default
#>   LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=English_United States.utf8
#> [2] LC_CTYPE=English_United States.utf8
#> [3] LC_MONETARY=English_United States.utf8
#> [4] LC_NUMERIC=C
#> [5] LC_TIME=English_United States.utf8
#>
#> time zone: America/New_York
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#> [1] patchwork_1.3.2 tidyr_1.3.2     dplyr_1.2.1     ggplot2_4.0.3
#>
#> loaded via a namespace (and not attached):
#>  [1] Matrix_1.7-5       gtable_0.3.6       compiler_4.6.1     tidyselect_1.2.1
#>  [5] splines_4.6.1      systemfonts_1.3.2  scales_1.4.0       textshaping_1.0.5
#>  [9] yaml_2.3.12        fastmap_1.2.0      lattice_0.22-9     R6_2.6.1
#> [13] labeling_0.4.3     generics_0.1.4     knitr_1.52         tibble_3.3.1
#> [17] pillar_1.11.1      RColorBrewer_1.1-3 rlang_1.3.0        xfun_0.60
#> [21] S7_0.2.2           otel_0.2.0         cli_3.6.6          withr_3.0.3
#> [25] magrittr_2.0.5     mgcv_1.9-4         digest_0.6.39      grid_4.6.1
#> [29] rstudioapi_0.19.0  nlme_3.1-169       lifecycle_1.0.5    vctrs_0.7.3
#> [33] evaluate_1.0.5     glue_1.8.1         farver_2.1.2       ragg_1.5.2
#> [37] rmarkdown_2.32     purrr_1.2.2        tools_4.6.1        pkgconfig_2.0.3
#> [41] htmltools_0.5.9
```

Submission notes

ChatGPT/Codex assisted with drafting and revising code, documentation and explanations. YouTube tutorials on GitHub workflows and data processing supported learning. This analysis companion accompanies the final narrative report.